UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

# CS2023 Algoritmos y Estructuras de Datos
# Representación de grafos
# Semana 11

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

20 de octubre de 2026

# Objetivos de la sesión



## Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Cómo llega un grafo en la entrada, y qué operaciones le piden los algoritmos

## Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Cómo llega un grafo en la entrada, y qué operaciones le piden los algoritmos
- Matriz de adyacencia y lista de adyacencia: memoria y costo de cada operación

## Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Cómo llega un grafo en la entrada, y qué operaciones le piden los algoritmos
- Matriz de adyacencia y lista de adyacencia: memoria y costo de cada operación
- Lista de aristas, grafo transpuesto y lista de adyacencia compacta

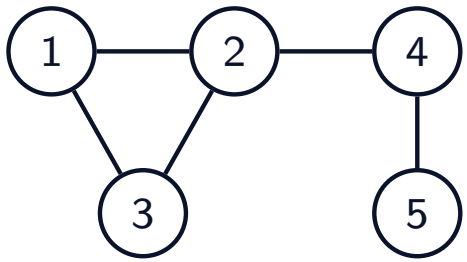
## Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Cómo llega un grafo en la entrada, y qué operaciones le piden los algoritmos
- Matriz de adyacencia y lista de adyacencia: memoria y costo de cada operación
- Lista de aristas, grafo transpuesto y lista de adyacencia compacta
- Grafos implícitos: grillas y estados, sin guardar el grafo

# Motivación

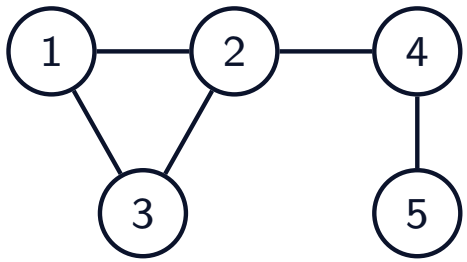
```
5 5
1 2
1 3
2 3
2 4
4 5
```




Primera línea: $n$ y $m$. Luego $m$ líneas $u$ $v$, una por arista (y un peso $w$, si lo hay).

# Motivación

```
5 5
1 2
1 3
2 3
2 4
4 5
```

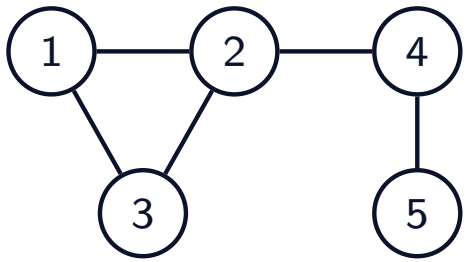



Primera línea: $n$ y $m$. Luego $m$ líneas $u$ $v$, una por arista (y un peso $w$, si lo hay).

¿Qué le piden al grafo los algoritmos que vienen?

## Motivación

```
5 5
1 2
1 3
2 3
2 4
4 5
```




Primera línea: $n$ y $m$. Luego $m$ líneas $u$ $v$, una por arista (y un peso $w$, si lo hay).

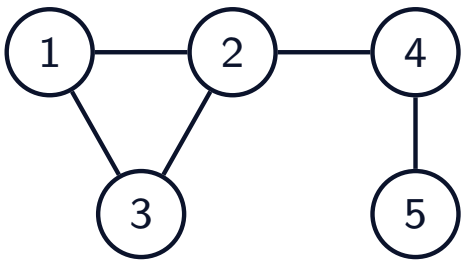
¿Qué le piden al grafo los algoritmos que vienen?

Sobre todo, tres cosas

Recorrer los vecinos de un vértice; preguntar si $u$ y $v$ son adyacentes; conocer el grado de un vértice. La forma de guardar el grafo decide cuánto cuesta cada una, y cuánta memoria se gasta.

# Matriz de adyacencia

# Matriz de adyacencia

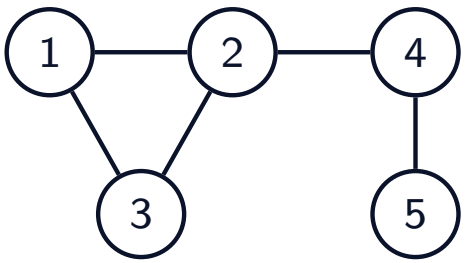



| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 | 0 |
| 2 | 1 | 0 | 1 | 1 | 0 |
| 3 | 1 | 1 | 0 | 0 | 0 |
| 4 | 0 | 1 | 0 | 0 | 1 |
| 5 | 0 | 0 | 0 | 1 | 0 |

## Definición

Una matriz $A$ de $n \times n$ con $A[u][v] = 1$ si $\{u, v\} \in E$ y $A[u][v] = 0$ si no.

# Matriz de adyacencia



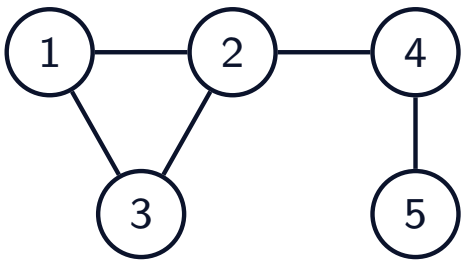

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 | 0 |
| 2 | 1 | 0 | 1 | 1 | 0 |
| 3 | 1 | 1 | 0 | 0 | 0 |
| 4 | 0 | 1 | 0 | 0 | 1 |
| 5 | 0 | 0 | 0 | 1 | 0 |

## Definición

Una matriz $A$ de $n \times n$ con $A[u][v] = 1$ si $\{u, v\} \in E$ y $A[u][v] = 0$ si no.

La fila $u$ dice quiénes son los vecinos de $u$: en azul, la fila 2, con unos en las columnas 1, 3 y 4.

# Matriz de adyacencia




| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 | 0 |
| 2 | 1 | 0 | 1 | 1 | 0 |
| 3 | 1 | 1 | 0 | 0 | 0 |
| 4 | 0 | 1 | 0 | 0 | 1 |
| 5 | 0 | 0 | 0 | 1 | 0 |

## Definición

Una matriz $A$ de $n \times n$ con $A[u][v] = 1$ si $\{u, v\} \in E$ y $A[u][v] = 0$ si no.

La fila $u$ dice quiénes son los vecinos de $u$: en azul, la fila 2, con unos en las columnas 1, 3 y 4.

En un grafo no dirigido, $A[u][v] = A[v][u]$: la matriz es simétrica, y la diagonal es cero si no hay lazos.

## Construir la matriz y usarla

**Algoritmo 1:** LEER-MATRIZ

leer $n$ y $m$; $A \leftarrow$ matriz de $n \times n$ llena de ceros
**para** $i \leftarrow 1$ **a** $m$ **hacer**
    leer $u$ y $v$
    $A[u][v] \leftarrow 1$, $A[v][u] \leftarrow 1$ `// no dirigido: las dos`

## Construir la matriz y usarla

**Algoritmo 2:** LEER-MATRIZ

leer $n$ y $m$; $A \leftarrow$ matriz de $n \times n$ llena de ceros
**para** $i \leftarrow 1$ **a** $m$ **hacer**
    leer $u$ y $v$
    $A[u][v] \leftarrow 1$, $A[v][u] \leftarrow 1$ `// no dirigido: las dos`

| **Operación** | **Costo** |
|---|---|
| ¿$u$ y $v$ son adyacentes? | $O(1)$: mirar $A[u][v]$ |
| recorrer los vecinos de $u$ | $\Theta(n)$: toda la fila, aunque tenga pocos unos |
| grado de $u$ | $\Theta(n)$: sumar la fila |
| agregar o quitar una arista | $O(1)$ |
| memoria | $\Theta(n^2)$ |

## Variantes de la matriz

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

### Con pesos

$A[u][v] = w(u, v)$, y un valor especial (por ejemplo $\infty$) cuando no hay arista.
Ojo: el 0 ya no sirve para decir "no hay arista" si hay pesos 0.

## Variantes de la matriz

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

### Con pesos

$A[u][v] = w(u, v)$, y un valor especial (por ejemplo $\infty$) cuando no hay arista. Ojo: el 0 ya no sirve para decir "no hay arista" si hay pesos 0.

### Dirigido

Solo se marca $A[u][v]$. La fila $u$ son las aristas que salen de $u$ y la columna $u$, las que llegan: el grado de entrada es la suma de una columna.

## ¿Cuánta memoria gasta la matriz?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Un grafo tiene $n = 10^5$ vértices y $m = 2 \cdot 10^5$ aristas. ¿Cabe su matriz de adyacencia?

## ¿Cuánta memoria gasta la matriz?

Un grafo tiene $n = 10^5$ vértices y $m = 2 \cdot 10^5$ aristas. ¿Cabe su matriz de adyacencia?

No

Son $n^2 = 10^{10}$ casillas: aunque cada una ocupe un solo byte, son 10 GB, para guardar $2 \cdot 10^5$ aristas. Casi toda la matriz son ceros.

## ¿Cuánta memoria gasta la matriz?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Un grafo tiene $n = 10^5$ vértices y $m = 2 \cdot 10^5$ aristas. ¿Cabe su matriz de adyacencia?

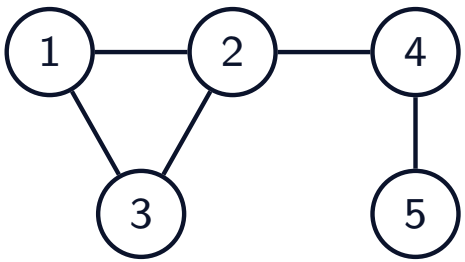
No

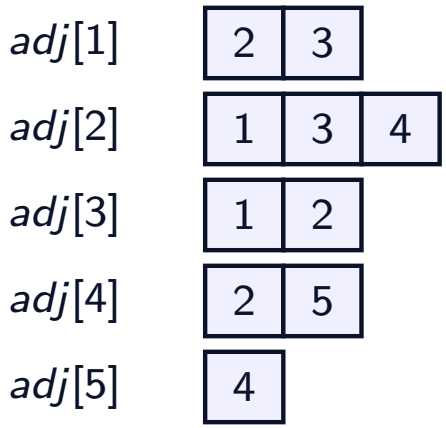
Son $n^2 = 10^{10}$ casillas: aunque cada una ocupe un solo byte, son 10 GB, para guardar $2 \cdot 10^5$ aristas. Casi toda la matriz son ceros.

La matriz conviene con $n$ de hasta unos pocos miles, sobre todo si el grafo es **denso** ($m$ del orden de $n^2$) o si se pregunta muchas veces si dos vértices son adyacentes.

# Lista de adyacencia

# Lista de adyacencia

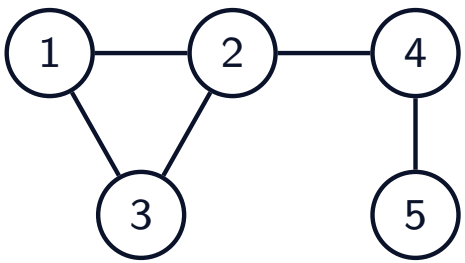


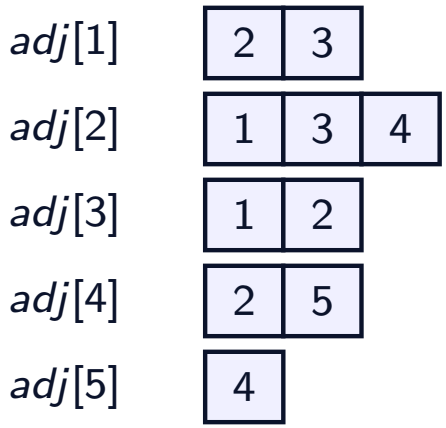




## Definición

Un arreglo de $n$ listas (vectores): $adj[u]$ guarda los vecinos de $u$, uno por cada arista incidente.

# Lista de adyacencia




## Definición

Un arreglo de $n$ listas (vectores): $adj[u]$ guarda los vecinos de $u$, uno por cada arista incidente.

Cada arista no dirigida aparece dos veces: $\{2, 4\}$ está en $adj[2]$ y en $adj[4]$. Por eso las listas suman $\sum_u |adj[u]| = \sum_u \deg(u) = 2m$ elementos: el lema del apretón de manos.

## Construir las listas

**Algoritmo 3:** LEER-LISTA

leer $n$ y $m$; $adj \leftarrow$ arreglo de $n$ vectores vacíos
**para** $i \leftarrow 1$ **a** $m$ **hacer**
    leer $u$ y $v$
    PUSH_BACK($adj[u], v$), PUSH_BACK($adj[v], u$)

## Construir las listas

**Algoritmo 4:** LEER-LISTA

leer $n$ y $m$; $adj \leftarrow$ arreglo de $n$ vectores vacíos
**para** $i \leftarrow 1$ **a** $m$ **hacer**
    leer $u$ y $v$
    PUSH_BACK($adj[u], v$), PUSH_BACK($adj[v], u$)

Los vecinos quedan en el orden de la entrada: al leer `1 2`, `1 3`, `2 3`, `2 4`, `4 5`, queda $adj[2] = [1, 3, 4]$. $O(n + m)$ en total: cada PUSH_BACK es $O(1)$ amortizado (semana 2).

# ¿Cuánto cuesta cada operación?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

| **Operación** | **Costo** |
|---|---|
| recorrer los vecinos de $u$ | $O(\deg u)$ |
| grado de $u$ | $O(1)$: el tamaño de $adj[u]$ |
| ¿$u$ y $v$ son adyacentes? | $O(\deg u)$: buscar $v$ en $adj[u]$ |
| agregar una arista | $O(1)$ amortizado |
| memoria | $\Theta(n+m)$ |

## ¿Cuánto cuesta cada operación?

| Operación | Costo |
|---|---|
| recorrer los vecinos de $u$ | $O(\deg u)$ |
| grado de $u$ | $O(1)$: el tamaño de $adj[u]$ |
| ¿$u$ y $v$ son adyacentes? | $O(\deg u)$: buscar $v$ en $adj[u]$ |
| agregar una arista | $O(1)$ amortizado |
| memoria | $\Theta(n+m)$ |

¿Cuánto cuesta recorrer los vecinos de **todos** los vértices?

## ¿Cuánto cuesta cada operación?

| Operación | Costo |
|---|---|
| recorrer los vecinos de $u$ | $O(\deg u)$ |
| grado de $u$ | $O(1)$: el tamaño de $adj[u]$ |
| ¿$u$ y $v$ son adyacentes? | $O(\deg u)$: buscar $v$ en $adj[u]$ |
| agregar una arista | $O(1)$ amortizado |
| memoria | $\Theta(n+m)$ |

¿Cuánto cuesta recorrer los vecinos de **todos** los vértices?

$\Theta(n+m)$, no $\Theta(n^2)$

$\sum_u (1+\deg u) = n+2m$. Con la matriz serían $n^2$ pasos siempre. Casi todos los algoritmos de grafos que vienen hacen justamente esto: por eso la lista es la representación por defecto.

## Quitar una arista de las listas

En $adj[u]$ los vecinos no tienen un orden especial. ¿Cómo se quita $v$ sin desplazar todo lo que viene después?

## Quitar una arista de las listas

En $adj[u]$ los vecinos no tienen un orden especial. ¿Cómo se quita $v$ sin desplazar todo lo que viene después?

**Algoritmo 6:** QUITAR-ARISTA$(u, v)$

$i \leftarrow$ la posición de $v$ en $adj[u]$ // $O(\deg u)$
intercambiar $adj[u][i]$ con el último de $adj[u]$, POP_BACK$(adj[u])$
$j \leftarrow$ la posición de $u$ en $adj[v]$ // $O(\deg v)$
intercambiar $adj[v][j]$ con el último de $adj[v]$, POP_BACK$(adj[v])$

## Quitar una arista de las listas

En $adj[u]$ los vecinos no tienen un orden especial. ¿Cómo se quita $v$ sin desplazar todo lo que viene después?

**Algoritmo 7:** QUITAR-ARISTA$(u, v)$

$i \leftarrow$ la posición de $v$ en $adj[u]$ // $O(\deg u)$
intercambiar $adj[u][i]$ con el último de $adj[u]$, POP_BACK$(adj[u])$
$j \leftarrow$ la posición de $u$ en $adj[v]$ // $O(\deg v)$
intercambiar $adj[v][j]$ con el último de $adj[v]$, POP_BACK$(adj[v])$

Como el orden no importa, el hueco se tapa con el último elemento: el borrado en sí es $O(1)$ y el costo es encontrar la posición, $O(\deg u + \deg v)$. En la matriz, quitar una arista es $O(1)$.

## ¿Y si también hay que preguntar si son adyacentes?

Buscar $v$ en $adj[u]$ cuesta $O(\deg u)$, y un vértice puede tener $10^5$ vecinos. ¿Cómo se responde en $O(1)$ sin gastar $\Theta(n^2)$?

## ¿Y si también hay que preguntar si son adyacentes?

Buscar $v$ en $adj[u]$ cuesta $O(\deg u)$, y un vértice puede tener $10^5$ vecinos. ¿Cómo se responde en $O(1)$ sin gastar $\Theta(n^2)$?

### Con una tabla hash (semana 5)

Se guardan las parejas $(u, v)$ en una tabla hash, con clave $u \cdot (n+1) + v$: la consulta cuesta $O(1)$ en promedio y la memoria sigue siendo $\Theta(n+m)$.

## ¿Y si también hay que preguntar si son adyacentes?

Buscar $v$ en $adj[u]$ cuesta $O(\deg u)$, y un vértice puede tener $10^5$ vecinos. ¿Cómo se responde en $O(1)$ sin gastar $\Theta(n^2)$?

### Con una tabla hash (semana 5)

Se guardan las parejas $(u, v)$ en una tabla hash, con clave $u \cdot (n+1) + v$: la consulta cuesta $O(1)$ en promedio y la memoria sigue siendo $\Theta(n+m)$.

Las listas se quedan para recorrer vecinos, y la tabla se usa solo para preguntar: cada estructura hace lo que sabe hacer bien.

## Pesos, sentido y el grafo transpuesto

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

### Con pesos y dirigidos

Con pesos, $adj[u]$ guarda pares $(v, w)$. En un grafo dirigido, la arista $u \to v$ solo se agrega a $adj[u]$: la lista tiene las aristas que **salen** de $u$.

# Pesos, sentido y el grafo transpuesto

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

## Con pesos y dirigidos

Con pesos, $adj[u]$ guarda pares $(v, w)$. En un grafo dirigido, la arista $u \to v$ solo se agrega a $adj[u]$: la lista tiene las aristas que **salen** de $u$.

**Algoritmo 9:** TRANSPUESTO($adj$) — invierte el sentido de todas las aristas

$adjT \leftarrow$ arreglo de $n$ vectores vacíos
**para** $u \leftarrow 1$ **a** $n$ **hacer**
  **para cada** $v \in adj[u]$ **hacer** PUSH_BACK($adjT[v], u$)
**devolver** $adjT$

# Pesos, sentido y el grafo transpuesto

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

## Con pesos y dirigidos

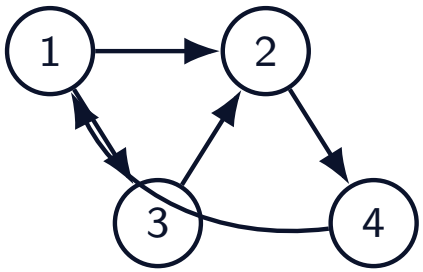
Con pesos, $adj[u]$ guarda pares $(v, w)$. En un grafo dirigido, la arista $u \to v$ solo se agrega a $adj[u]$: la lista tiene las aristas que **salen** de $u$.

**Algoritmo 10:** TRANSPUESTO($adj$) — invierte el sentido de todas las aristas

$adjT \leftarrow$ arreglo de $n$ vectores vacíos
**para** $u \leftarrow 1$ **a** $n$ **hacer**
  **para cada** $v \in adj[u]$ **hacer** PUSH_BACK($adjT[v], u$)
**devolver** $adjT$

$O(n + m)$. En $adjT[v]$ quedan los vértices desde los que se **llega** a $v$: sirve para preguntar "¿de quién depende $v$?".

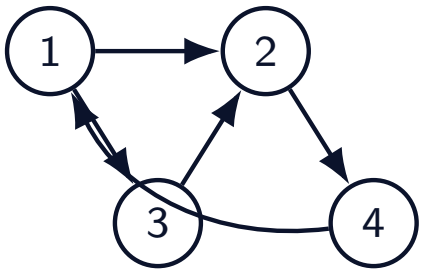
# Un grafo dirigido, en cada representación




| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 |
| 2 | 0 | 0 | 0 | 1 |
| 3 | 0 | 1 | 0 | 0 |
| 4 | 1 | 0 | 0 | 0 |

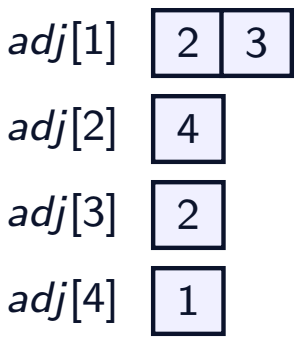
La matriz ya no es simétrica: $A[1][2] = 1$ pero $A[2][1] = 0$.

# Un grafo dirigido, en cada representación




| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 |
| 2 | 0 | 0 | 0 | 1 |
| 3 | 0 | 1 | 0 | 0 |
| 4 | 1 | 0 | 0 | 0 |

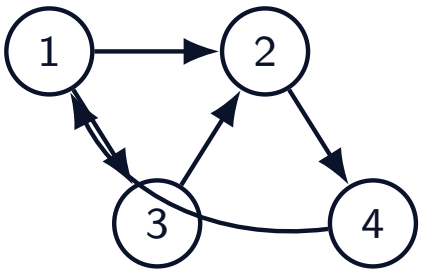
salen



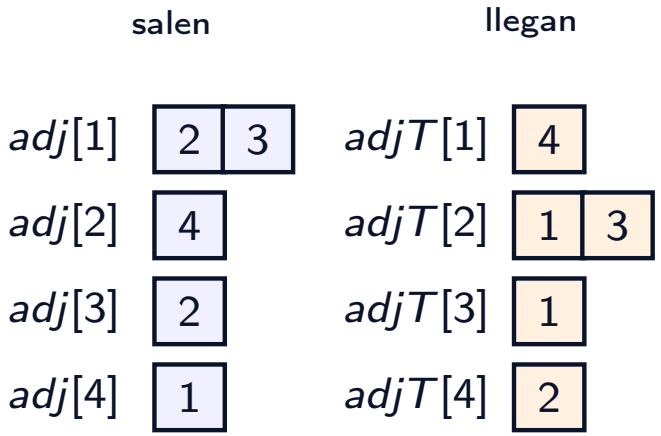

La matriz ya no es simétrica: $A[1][2] = 1$ pero $A[2][1] = 0$.

Cada arista aparece una sola vez en $adj$: $\sum_u |adj[u]| = m = 5$. El tamaño de $adj[u]$ es el grado de salida.

# Un grafo dirigido, en cada representación




| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 |
| 2 | 0 | 0 | 0 | 1 |
| 3 | 0 | 1 | 0 | 0 |
| 4 | 1 | 0 | 0 | 0 |




La matriz ya no es simétrica: $A[1][2] = 1$ pero $A[2][1] = 0$.

Cada arista aparece una sola vez en $adj$: $\sum_u |adj[u]| = m = 5$. El tamaño de $adj[u]$ es el grado de salida.

TRANSPUESTO recorre $u = 1, \ldots, 4$ y deja en $adjT[v]$ los vértices desde los que se llega a $v$: su tamaño es el grado de entrada.

# Lista de aristas

## Lista de aristas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

**Definición**

Un arreglo con las $m$ aristas tal como llegan: triplas $(u, v, w)$. Memoria $\Theta(m)$.

## Lista de aristas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

**Definición**

Un arreglo con las $m$ aristas tal como llegan: triplas $(u, v, w)$. Memoria $\Theta(m)$.

Sirve cuando se procesan las aristas una por una

Por ejemplo, de menor a mayor peso: basta ordenar el arreglo. Para recorrer los vecinos de un vértice es mala: hay que revisar las $m$ aristas.

## Lista de aristas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

**Definición**

Un arreglo con las $m$ aristas tal como llegan: triplas $(u, v, w)$. Memoria $\Theta(m)$.

Sirve cuando se procesan las aristas una por una

Por ejemplo, de menor a mayor peso: basta ordenar el arreglo. Para recorrer los vecinos de un vértice es mala: hay que revisar las $m$ aristas.

Es fácil pasar de una representación a otra: de la lista de aristas a la de adyacencia en $O(n + m)$, y de la matriz a las listas en $\Theta(n^2)$.

# ¿Cuál elegir?

## Las tres representaciones

| | Matriz | Lista de adyacencia | Lista de aristas |
|---|---|---|---|
| memoria | $\Theta(n^2)$ | $\Theta(n+m)$ | $\Theta(m)$ |
| ¿adyacentes? | $O(1)$ | $O(\deg u)$ | $O(m)$ |
| vecinos de $u$ | $\Theta(n)$ | $O(\deg u)$ | $O(m)$ |

## Las tres representaciones

| | Matriz | Lista de adyacencia | Lista de aristas |
|---|---|---|---|
| memoria | $\Theta(n^2)$ | $\Theta(n+m)$ | $\Theta(m)$ |
| ¿adyacentes? | $O(1)$ | $O(\deg u)$ | $O(m)$ |
| vecinos de $u$ | $\Theta(n)$ | $O(\deg u)$ | $O(m)$ |
| todos los vecinos | $\Theta(n^2)$ | $\Theta(n+m)$ | $\Theta(m)$ |
| grado de $u$ | $\Theta(n)$ | $O(1)$ | $O(m)$ |
| agregar arista | $O(1)$ | $O(1)$ amort. | $O(1)$ amort. |

## Las tres representaciones

| | Matriz | Lista de adyacencia | Lista de aristas |
|---|---|---|---|
| memoria | $\Theta(n^2)$ | $\Theta(n+m)$ | $\Theta(m)$ |
| ¿adyacentes? | $O(1)$ | $O(\deg u)$ | $O(m)$ |
| vecinos de $u$ | $\Theta(n)$ | $O(\deg u)$ | $O(m)$ |
| todos los vecinos | $\Theta(n^2)$ | $\Theta(n+m)$ | $\Theta(m)$ |
| grado de $u$ | $\Theta(n)$ | $O(1)$ | $O(m)$ |
| agregar arista | $O(1)$ | $O(1)$ amort. | $O(1)$ amort. |

### Por defecto: lista de adyacencia

Los algoritmos recorren vecinos, y con $n = 10^5$ es la única que cabe. La matriz, con $n$ de pocos miles y muchas consultas de adyacencia; la lista de aristas, para procesar aristas en orden.

## Ejemplo: vecinos en común

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

**¿Cuántos vecinos en común tienen $u$ y $v$?**

Con la matriz: recorrer las dos filas y contar las columnas con uno en ambas, $\Theta(n)$. ¿Se puede hacer mejor con las listas?

## Ejemplo: vecinos en común

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

**¿Cuántos vecinos en común tienen $u$ y $v$?**

Con la matriz: recorrer las dos filas y contar las columnas con uno en ambas, $\Theta(n)$. ¿Se puede hacer mejor con las listas?

**Algoritmo 12:** VECINOS-COMUNES$(u, v)$ — *marca* empieza en FALSO

```
cnt ← 0
para cada x ∈ adj[u] hacer marca[x] ← VERDADERO
para cada y ∈ adj[v] hacer
    si marca[y] entonces cnt ← cnt + 1
para cada x ∈ adj[u] hacer marca[x] ← FALSO
devolver cnt
```

## Ejemplo: vecinos en común

**¿Cuántos vecinos en común tienen $u$ y $v$?**

Con la matriz: recorrer las dos filas y contar las columnas con uno en ambas, $\Theta(n)$. ¿Se puede hacer mejor con las listas?

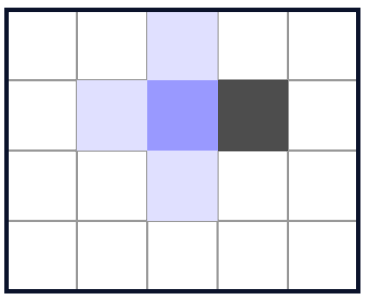
**Algoritmo 13:** VECINOS-COMUNES$(u, v)$ — *marca* empieza en FALSO

```
cnt ← 0
para cada x ∈ adj[u] hacer marca[x] ← VERDADERO
para cada y ∈ adj[v] hacer
    si marca[y] entonces cnt ← cnt + 1
para cada x ∈ adj[u] hacer marca[x] ← FALSO
devolver cnt
```

$O(\deg u + \deg v)$ por consulta

# Grafos implícitos

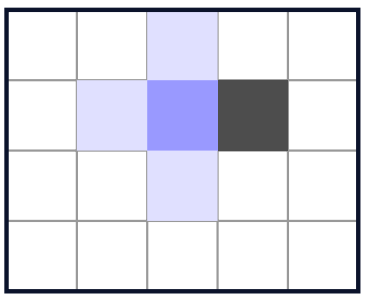
# Grillas: vecinos calculados al vuelo



**Algoritmo 14:** VECINOS$(f, c)$ — grilla de $R \times C$

$df \leftarrow [-1, 1, 0, 0]$, $dc \leftarrow [0, 0, -1, 1]$
**para** $k \leftarrow 0$ **a** $3$ **hacer**
  $f' \leftarrow f + df[k]$, $c' \leftarrow c + dc[k]$
  **si** $0 \le f' < R$ **y** $0 \le c' < C$ **y** $(f', c')$ *no es pared*
    **entonces**
      reportar $(f', c')$

# Grillas: vecinos calculados al vuelo

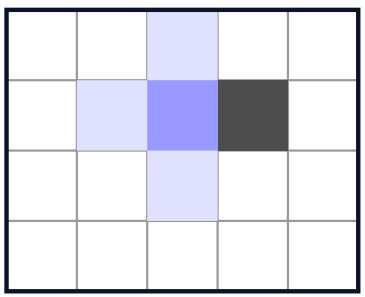


**Algoritmo 15:** VECINOS$(f, c)$ — grilla de $R \times C$

$df \leftarrow [-1, 1, 0, 0]$, $dc \leftarrow [0, 0, -1, 1]$
**para** $k \leftarrow 0$ **a** $3$ **hacer**
  $f' \leftarrow f + df[k]$, $c' \leftarrow c + dc[k]$
  **si** $0 \le f' < R$ **y** $0 \le c' < C$ **y** $(f', c')$ *no es pared*
    **entonces**
      reportar $(f', c')$

No se guarda ningún grafo: los vecinos se calculan cuando se piden. A la izquierda, la casilla $(1, 2)$ tiene tres vecinos; el cuarto es pared.

# Grillas: vecinos calculados al vuelo



**Algoritmo 16:** VECINOS$(f, c)$ — grilla de $R \times C$

$df \leftarrow [-1, 1, 0, 0]$, $dc \leftarrow [0, 0, -1, 1]$
**para** $k \leftarrow 0$ **a** $3$ **hacer**
  $f' \leftarrow f + df[k]$, $c' \leftarrow c + dc[k]$
  **si** $0 \le f' < R$ **y** $0 \le c' < C$ **y** $(f', c')$ *no es pared*
    **entonces**
      reportar $(f', c')$

No se guarda ningún grafo: los vecinos se calculan cuando se piden. A la izquierda, la casilla $(1, 2)$ tiene tres vecinos; el cuarto es pared.

Para usar arreglos indexados por vértice, la casilla $(f, c)$ se numera $f \cdot C + c$: de 0 a $RC - 1$.

## Grafos de estados

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

### Jarras de 3 y 5 litros

Un vértice es un estado $(a, b)$: los litros en cada jarra. Las aristas son las jugadas: llenar una jarra, vaciarla, o pasar agua de una a otra hasta que una se vacíe o la otra se llene.

## Grafos de estados

### Jarras de 3 y 5 litros

Un vértice es un estado $(a, b)$: los litros en cada jarra. Las aristas son las jugadas: llenar una jarra, vaciarla, o pasar agua de una a otra hasta que una se vacíe o la otra se llene.

### El grafo tampoco se guarda

Hay $(3+1)(5+1) = 24$ estados posibles, pero los vecinos de un estado se generan aplicando las seis jugadas. La pregunta "¿cómo obtener 4 litros con la menor cantidad de jugadas?" es otra vez una distancia en un grafo.

# Lista de adyacencia compacta

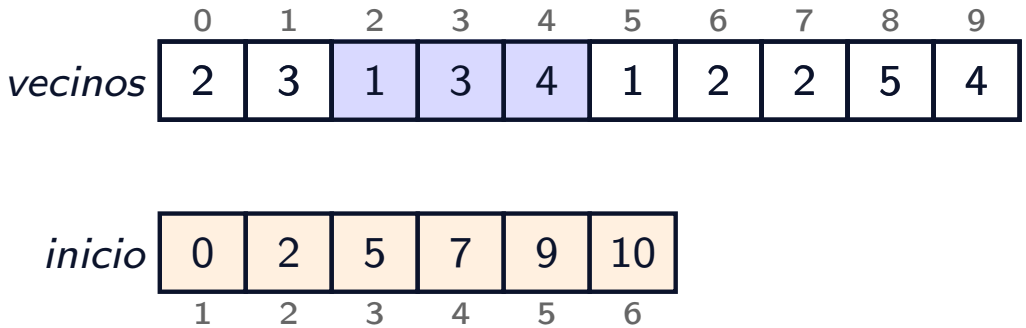
## Todas las listas en un solo arreglo




### La idea

Poner las listas $adj[1], adj[2], \ldots, adj[n]$ una detrás de otra en un solo arreglo *vecinos* de $2m$ casillas, y guardar dónde empieza cada una en *inicio*.

# Todas las listas en un solo arreglo




## La idea

Poner las listas $adj[1], adj[2], \ldots, adj[n]$ una detrás de otra en un solo arreglo *vecinos* de $2m$ casillas, y guardar dónde empieza cada una en *inicio*.

Los vecinos de $u$ son $vecinos[inicio[u] \ldots inicio[u+1]-1]$: los de 2 son las casillas 2 a 4, es decir, 1, 3 y 4.

## Construir la lista compacta

**Algoritmo 17:** CONSTRUIR-COMPACTA — con la lista de aristas

$grado[u] \leftarrow 0$ para todo $u$
**para cada** *arista* $(u, v)$ **hacer** $grado[u] \leftarrow grado[u] + 1$,
$grado[v] \leftarrow grado[v] + 1$
$inicio[1] \leftarrow 0$
**para** $u \leftarrow 1$ **a** $n$ **hacer** $inicio[u+1] \leftarrow inicio[u] + grado[u]$
$pos[u] \leftarrow inicio[u]$ para todo $u$
**para cada** *arista* $(u, v)$ **hacer**
  $vecinos[pos[u]] \leftarrow v$, $pos[u] \leftarrow pos[u] + 1$
  $vecinos[pos[v]] \leftarrow u$, $pos[v] \leftarrow pos[v] + 1$

## Construir la lista compacta

**Algoritmo 18:** CONSTRUIR-COMPACTA — con la lista de aristas

$grado[u] \leftarrow 0$ para todo $u$
**para cada** *arista* $(u, v)$ **hacer** $grado[u] \leftarrow grado[u] + 1$,
$grado[v] \leftarrow grado[v] + 1$
$inicio[1] \leftarrow 0$
**para** $u \leftarrow 1$ **a** $n$ **hacer** $inicio[u+1] \leftarrow inicio[u] + grado[u]$
$pos[u] \leftarrow inicio[u]$ para todo $u$
**para cada** *arista* $(u, v)$ **hacer**
  $vecinos[pos[u]] \leftarrow v$, $pos[u] \leftarrow pos[u] + 1$
  $vecinos[pos[v]] \leftarrow u$, $pos[v] \leftarrow pos[v] + 1$

Dos pasadas por las aristas: $O(n + m)$. Misma memoria que las listas, pero en dos arreglos contiguos y sin vectores que crecen. Sirve cuando el grafo no cambia después de leerlo.

## Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Matriz: $\Theta(n^2)$ de memoria, adyacencia en $O(1)$, vecinos en $\Theta(n)$. Solo para $n$ de unos pocos miles.
- Lista de adyacencia: $\Theta(n+m)$, vecinos en $O(\deg u)$ y todos los vecinos en $\Theta(n+m)$: la opción por defecto. Con una tabla hash, la adyacencia también sale en $O(1)$.

## Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Matriz: $\Theta(n^2)$ de memoria, adyacencia en $O(1)$, vecinos en $\Theta(n)$. Solo para $n$ de unos pocos miles.
- Lista de adyacencia: $\Theta(n+m)$, vecinos en $O(\deg u)$ y todos los vecinos en $\Theta(n+m)$: la opción por defecto. Con una tabla hash, la adyacencia también sale en $O(1)$.
- Lista de aristas para procesarlas en orden; transpuesto en $O(n+m)$; lista compacta con grados y posiciones de inicio.
- Grillas y grafos de estados: los vecinos se calculan al vuelo, sin guardar el grafo.

# Logros de la sesión

## Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos construir la matriz y las listas de adyacencia a partir de la entrada.

## Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos construir la matriz y las listas de adyacencia a partir de la entrada.
- Sabemos el costo de cada operación en cada representación, y elegir la adecuada según $n$, $m$ y las consultas.

## Logros de la sesión

- Sabemos construir la matriz y las listas de adyacencia a partir de la entrada.
- Sabemos el costo de cada operación en cada representación, y elegir la adecuada según $n$, $m$ y las consultas.
- Sabemos construir el grafo transpuesto y la lista compacta.

## Logros de la sesión

- Sabemos construir la matriz y las listas de adyacencia a partir de la entrada.
- Sabemos el costo de cada operación en cada representación, y elegir la adecuada según $n$, $m$ y las consultas.
- Sabemos construir el grafo transpuesto y la lista compacta.
- Sabemos trabajar con grafos implícitos, como grillas y estados.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

# Gracias